Snayu: Neural Network Framework from Scratch

Step-by-Step Implementation Guide in Pure Python & NumPy

OVERVIEW 1. Core Design Philosophy

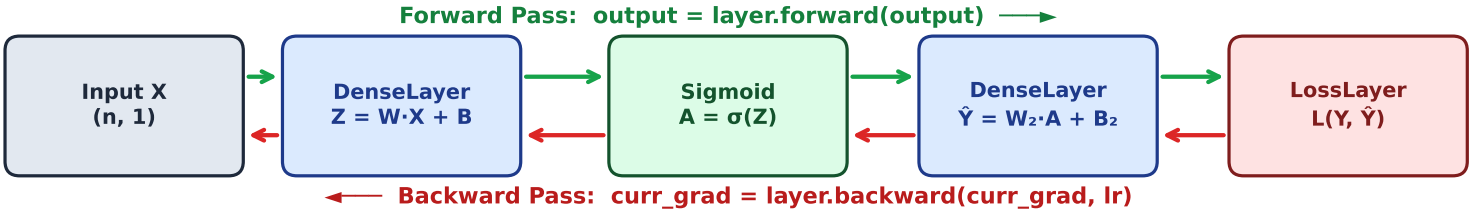
- Snayu treats every neural network building block as an independent, self-contained Layer.
- Linear transformations (DenseLayer) and non-linear activations (ActivationLayer) are separate layers.
- Every layer obeys a single contract: forward(inputs) computes outputs; backward(grad, lr) computes gradients.
- A neural network is simply a Python list of layer instances: network = [DenseLayer(...), Sigmoid(), ...].

STRUCTURE 2. Project Folder & File Hierarchy

FILE: Project Directory Layout

```
Snayu/
├── Layer/                      # Core abstract & concrete layer classes
│   ├── BaseLayer.py            # Step 1: Abstract base class (Layer interface)
│   ├── DenseLayer.py           # Step 2: Fully-connected linear layer (Z = W*X + B)
│   ├── ActivationLayer.py      # Step 3: Generic wrapper for activation functions
│   └── LossLayer.py            # Step 5: Generic wrapper for loss functions
├── Functions/                  # Concrete mathematical functions
│   ├── ActivationFunctions.py  # Step 4: Sigmoid(ActivationLayer) implementation
│   └── LossFunctions.py        # Step 6: MSE(LossLayer) implementation
├── Network/                    # High-level execution & training engine
│   ├── predict.py              # Step 7a: Forward pass through the network list
│   └── train.py                # Step 7b: SGD training loop, accuracy & plotting
└── Examples/
    └── xor.py                  # Step 8: End-to-end XOR training & evaluation
```

DATAFLOW 3. Forward & Backward Propagation Flow



Shape Convention Used Throughout Snayu

• Every single sample x is a 2D column vector of shape (input_size, 1) — NOT a 1D array (input_size,).
• Why? Because matrix multiplication np.dot(W, X) with W of shape (m, n) and X of shape (n, 1) cleanly produces Z of shape (m, 1), and np.dot(dL_dZ, X.T) produces a weight gradient of shape (m, n)!
• Therefore, a dataset of N samples is shaped (N, input_size, 1), e.g., (4, 2, 1) for XOR.

Step 1 & Step 2: BaseLayer and DenseLayer

Defining the Layer Interface and Fully-Connected Linear Math

STEP 1 Create the Abstract Base Layer (Layer/BaseLayer.py)

- Before writing any math, define the base class Layer using Python's abc (Abstract Base Class) module.
- Marking forward() and backward() with @abstractmethod guarantees every subclass implements both methods.

FILE: Layer/BaseLayer.py

```
1  | from abc import ABC, abstractmethod
2  |
3  | class Layer(ABC):
4  |     @abstractmethod
5  |     def forward(self, inputs):
6  |         pass
7  |
8  |     @abstractmethod
9  |     def backward(self, grad_output, learning_rate):
10 |         pass
```

STEP 2 Implement the Fully-Connected Layer (Layer/DenseLayer.py)

Mathematical Equations for DenseLayer (Input size n → Output size m)

1. Parameters: Weights W ∈ R^(m × n) initialized with small random Gaussian values; Biases B ∈ R^(m × 1) = 0.
2. Forward Pass: Z = W · X + B (where X has shape (n, 1) and output Z has shape (m, 1)).
3. Backward Pass (given incoming gradient dL/dZ of shape (m, 1) from the next layer):
   - Weight Gradient: dL/dW = (dL/dZ) · X^T → shape (m, 1) × (1, n) = (m, n)
   - Bias Gradient: dL/dB = dL/dZ → shape (m, 1)
   - Input Gradient: dL/dX = W^T · (dL/dZ) → shape (n, m) × (m, 1) = (n, 1) (passed backward)
   - Parameter Update: W ← W - lr * (dL/dW), B ← B - lr * (dL/dB)

FILE: Layer/DenseLayer.py

```
1  | import numpy as np
2  | from Layer.BaseLayer import Layer
3  |
4  | class DenseLayer(Layer):
5  |     def __init__(self, input_size, output_size):
6  |         self.weights = np.random.randn(output_size, input_size) * 0.01  # W(m,n)
7  |         self.biases = np.zeros((output_size, 1))                        # B(m,1)
8  |
9  |     def forward(self, inputs):
10 |         self.inputs = inputs
11 |         return np.dot(self.weights, self.inputs) + self.biases          # Z = WX + B -> (m,1)
12 |
13 |     def backward(self, prev_layer_ret_grad, learning_rate):
14 |         # 1. Calculate gradients for trainable parameters
15 |         weights_grad = np.dot(prev_layer_ret_grad, self.inputs.T)       # dL/dW = dL/dZ * X^T
16 |         biases_grad = prev_layer_ret_grad                               # dL/dB = dL/dZ
17 |
18 |         # 2. Calculate gradient wrt input X BEFORE updating self.weights!
19 |         curr_layer_ret_grad = np.dot(self.weights.T, prev_layer_ret_grad) # dL/dX = W^T * dL/dZ
20 |
21 |         # 3. Update weights and biases via Gradient Descent
22 |         self.weights -= learning_rate * weights_grad
23 |         self.biases -= learning_rate * biases_grad
24 |
25 |         return curr_layer_ret_grad
```

- Critical Detail #1: self.inputs = inputs in forward() caches X so backward() can compute dL/dZ · X^T.
- Critical Detail #2: curr_layer_ret_grad MUST be computed before self.weights -= ... so it uses the old W.

Step 3 & Step 4: Activations & Sigmoid

Decoupling Non-Linearity into a Reusable ActivationLayer

STEP 3 Create the Generic Activation Layer (Layer/ActivationLayer.py)

- Without non-linear activation functions, stacking multiple DenseLayers collapses into a single linear layer.
- Instead of hardcoding Sigmoid inside DenseLayer, ActivationLayer accepts any function f(x) and derivative f'(x).
- During backward(), by the Chain Rule: dL/dZ = (dL/dA) ⊙ f'(Z), where ⊙ is element-wise multiplication (*).

FILE: Layer/ActivationLayer.py

```
1  | import numpy as np
2  | from Layer.BaseLayer import Layer
3  |
4  | class ActivationLayer(Layer):
5  |     def __init__(self, activaion_func, activation_func_prime):
6  |         self.activation = activaion_func              # f(x) stored as a callable attribute
7  |         self.activation_prime = activation_func_prime # f'(x) stored as a callable attribute
8  |
9  |     def forward(self, input):
10 |         self.input = input                            # Cache Z -> (m,1) for backward pass
11 |         return self.activation(self.input)            # Return A = f(Z)
12 |
13 |     def backward(self, prev_layer_ret_grad, learning_rate):
14 |         # Element-wise multiplication (Hadamard product): dL/dZ = dL/dA * f'(Z)
15 |         return prev_layer_ret_grad * self.activation_prime(self.input)
```

STEP 4 Implement Sigmoid Activation (Functions/ActivationFunctions.py)

Sigmoid Function & Its Derivative

- Forward formula: σ(x) = 1 / (1 + e^(-x)) (squashes any real number into (0, 1))
- Derivative formula: σ'(x) = σ(x) · (1 - σ(x))
- Sigmoid subclasses ActivationLayer and passes both inner functions to super().__init__(...).

FILE: Functions/ActivationFunctions.py

```
1  | import numpy as np
2  | from Layer.ActivationLayer import ActivationLayer
3  |
4  | class Sigmoid(ActivationLayer):
5  |     def __init__(self):
6  |         def sigmoid(x):
7  |             return 1 / (1 + np.exp(-x))
8  |
9  |         def sigmoid_prime(x):
10 |             s = 1 / (1 + np.exp(-x))
11 |             return s * (1 - s)
12 |
13 |         super().__init__(sigmoid, sigmoid_prime)  # Hand over both functions to ActivationLayer
```

How Inheritance Connects Step 3 and Step 4

- When you write Sigmoid() in your network list, Python runs Sigmoid.__init__(self).
- Sigmoid.__init__ defines sigmoid(x) and sigmoid_prime(x) and calls super().__init__(...).
- Because Sigmoid inherits from ActivationLayer, calling layer.forward(Z) and layer.backward(grad, lr) automatically runs ActivationLayer.forward and ActivationLayer.backward using the Sigmoid formulas!

Step 5 & Step 6: LossLayer and Mean Squared Error

Measuring Prediction Error and Starting Backpropagation

STEP 5 Build the Generic Loss Wrapper (Layer/LossLayer.py)

- Just like ActivationLayer wraps any activation function, LossLayer wraps any loss function L(y, ŷ) and dL/dŷ.
- LossLayer.forward(inputs, target) computes the scalar loss value for logging and plotting.
- LossLayer.backward(inputs, target) computes the initial gradient dL/dŶ that kicks off backpropagation.

FILE: Layer/LossLayer.py

```
from Layer.BaseLayer import Layer
import numpy as np

class LossLayer(Layer):
    def __init__(self, loss_func, loss_func_prime):
        self.loss = loss_func              # Loss function L(y_true, y_pred)
        self.loss_prime = loss_func_prime  # Derivative dL/dy_pred
        self.inputs = None
        self.target = None

    def forward(self, inputs, target):
        if target is None: raise ValueError('LossLayer.forward() requires a target value.')
        if inputs is None: raise ValueError('LossLayer.forward() requires input values.')
        self.inputs = inputs
        self.target = target
        return self.loss(self.target, self.inputs)

    def backward(self, inputs, target):
        if target is None: raise ValueError('LossLayer.backward() requires a target value.')
        if inputs is None: raise ValueError('LossLayer.backward() requires input values.')
        self.inputs = inputs
        self.target = target
        return self.loss_prime(self.target, self.inputs)  # Returns initial gradient dL/dY
```

STEP 6 Implement Mean Squared Error (Functions/LossFunctions.py)

Mean Squared Error (MSE) Math

- Loss Formula: MSE(y, ŷ) = (1 / N) * Σ (y_i - ŷ_i)^2 = np.mean((y_true - y_pred) ** 2)
- Gradient wrt ŷ: d(MSE)/dŷ = (2 / N) * (ŷ - y) = 2 * (y_pred - y_true) / y_true.size
- Note the sign in mse_prime: (y_pred - y_true), NOT (y_true - y_pred)!

FILE: Functions/LossFunctions.py

```
import numpy as np
from Layer.LossLayer import LossLayer

class MSE(LossLayer):
    def __init__(self):
        def mse(y_true, y_pred):
            return np.mean((y_true - y_pred) ** 2)

        def mse_prime(y_true, y_pred):
            return 2 * (y_pred - y_true) / y_true.size

        super().__init__(mse, mse_prime)
```

Step 7: Network Prediction & Training Engine

Connecting Layers into a Forward Pass and Backpropagation Loop

STEP 7A Implement Forward Inference (Network/predict.py)

- predict(network, inputs) feeds the input column vector into layer 0, passes its output to layer 1, and so on.
- Because each layer caches its own input during layer.forward(output), predict() also prepares the network for backprop!

FILE: Network/predict.py

```
def predict(network, inputs):
    output = inputs
    for layer in network:
        output = layer.forward(output)  # Every layer's output becomes the next layer's input
    return output
```

STEP 7B Implement the Training Loop & Plotting (Network/train.py)

- _is_correct(y, out) checks if a prediction matches the target (threshold 0.5 for 1 output neuron, argmax for multi-class).
- For each epoch and each (x, y) sample: run predict(), compute loss, get loss gradient, and loop over reversed(network).
- After all epochs finish, plot_history(history) renders the Loss and Accuracy curves.

FILE: Network/train.py (Core Training & Evaluation Logic)

```
import numpy as np
import matplotlib.pyplot as plt
from Network.predict import predict

def _is_correct(y, out):
    y, out = np.asarray(y).ravel(), np.asarray(out).ravel()
    if len(out) > 1:
        pred = np.argmax(out)
        true = np.argmax(y) if len(y) > 1 else y[0]
    else:
        pred = 1 if out[0] >= 0.5 else 0
        true = y[0]
    return pred == true

def train(network, loss_layer, x_train, y_train,
          epochs=10, learning_rate=0.01, verbose=True, plot=True):
    sample_count = len(x_train)
    if sample_count == 0: raise ValueError('Training sample can not be empty !')
    if epochs < 1: raise ValueError('epoch must be greater than zero')

    history = {'loss': [], 'accuracy': []}
    for epoch in range(1, epochs + 1):
        total_loss, correct = 0.0, 0
        for x, y in zip(x_train, y_train):
            output = predict(network, x)                        # 1. Forward pass
            total_loss += loss_layer.forward(output, y)         # 2. Compute sample loss
            correct += _is_correct(y, output)                   # 3. Track accuracy

            curr_grad = loss_layer.backward(output, y)          # 4. Initial gradient dL/dY
            for layer in reversed(network):                     # 5. Backpropagate in reverse!
                curr_grad = layer.backward(curr_grad, learning_rate)

        avg_loss = total_loss / sample_count
        acc = correct / sample_count
        history['loss'].append(avg_loss)
        history['accuracy'].append(acc)
        if verbose:
            print(f'Epoch {epoch}/{epochs}  loss: {avg_loss:.6f}  accuracy: {acc:.4f}')

    if plot: plot_history(history)
    return history
```

Step 8: Solving the XOR Problem (Examples/xor.py)

Building, Training, and Testing a Multi-Layer Perceptron End-to-End

STEP 8 Write the XOR Example Script (Examples/xor.py)

- The XOR truth table ([0,0]→0, [0,1]→1, [1,0]→1, [1,1]→0) is not linearly separable — a single layer cannot solve it.
- We reshape X to (4, 2, 1) and Y to (4, 1, 1) so each sample x has shape (2, 1) and each target y has shape (1, 1).
- Each item inside network = [...] calls its class constructor (__init__) and creates a live object instance in the list.

FILE: Examples/xor.py

```
import os, sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import numpy as np
from Layer.DenseLayer import DenseLayer
from Functions.ActivationFunctions import Sigmoid
from Functions.LossFunctions import MSE
from Network.train import train
from Network.predict import predict

# 1. Prepare XOR dataset: 4 samples, each input is (2, 1) and target is (1, 1)
X = np.reshape([[0, 0], [0, 1], [1, 0], [1, 1]], (4, 2, 1))
Y = np.reshape([[0], [1], [1], [0]], (4, 1, 1))

# 2. Build network: each call creates a distinct layer instance in the list
np.random.seed(0)
network = [
    DenseLayer(2, 3),
    Sigmoid(),
    DenseLayer(3, 1),
    Sigmoid()
]

# 3. Train with MSE loss
loss_layer = MSE()
history = train(network, loss_layer, X, Y, epochs=20000, learning_rate=0.1)

# 4. Verify predictions on all 4 XOR inputs
for x, y in zip(X, Y):
    out = predict(network, x)[0, 0]
    pred = 1 if out >= 0.5 else 0
    print(f'Input: {x.ravel().tolist()} -> Pred: {out:.4f} (Class: {pred}) | Target: {y[0, 0]}')
```

VERIFY How to Run & Expected Console Output

FILE: Terminal Command & Output

```
$ python Examples/xor.py
Epoch 1/20000      loss: 0.250064  accuracy: 0.5000
...
Epoch 20000/20000  loss: 0.000718  accuracy: 1.0000

--- XOR Predictions ---
Input: [0, 0] -> Pred: 0.0230 (Class: 0) | Target: 0
Input: [0, 1] -> Pred: 0.9746 (Class: 1) | Target: 1
Input: [1, 0] -> Pred: 0.9746 (Class: 1) | Target: 1
Input: [1, 1] -> Pred: 0.0324 (Class: 0) | Target: 0
```

Quick Checklist When Extending Snayu With New Layers / Functions

- Want Tanh or ReLU? Add a class in Functions/ActivationFunctions.py that subclasses ActivationLayer.
- Want Binary Cross-Entropy? Add a class in Functions/LossFunctions.py that subclasses LossLayer.
- Want deeper networks? Add more DenseLayer(in, out) + Activation pairs to the network = [...] list!